

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION WITH OPENCV

COURSE CODE: DSA0204

COURSE OUTCOME COVERED

CO5: *Understand video processing - motion computation - 3D vision - geometry.*
(BTL 6)

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

Code of Conduct:

*I _Chalamala Sai Harshitha (192424346)_ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: C. Sai Harshitha

1. Motion Detection and Tracking System

A surveillance system must:

- A. Read video input
- B. Detect motion between consecutive frames
- C. Track moving objects
- D. Highlight detected motion regions

Constraints:

- Should handle dynamic background
- Must operate in near real-time

Design pseudocode for the system, including frame differencing, filtering, tracking logic, and visualization. Ensure proper control flow and handling of continuous video streams.

Input: Continuous video stream from camera

Output: Video frames with detected and tracked moving objects highlighted

Pseudocode

BEGIN

1. Initialize video capture from the camera.
2. Initialize the background model for handling dynamic background.
3. Initialize the object tracker.
4. Read the first frame from the video.
5. **WHILE** video stream is available **DO**
 - 5.1 Read the current frame.
 - 5.2 Resize the frame if required for near-real-time processing.
 - 5.3 Convert the current frame to grayscale.
 - 5.4 Compare the current frame with the background model or previous frame to obtain the frame difference.
 - 5.5 Apply filtering to remove noise from the difference image.
 - 5.6 Apply thresholding to identify significant motion regions.
 - 5.7 Apply morphological operations to remove small unwanted regions and fill gaps.
 - 5.8 Find contours of the detected motion regions.
 - 5.9 **FOR EACH** detected contour:
 - Calculate its bounding region.

- Ignore regions smaller than the minimum required size.
- Store the valid bounding region.

5.10 **IF** valid motion regions are detected **THEN**

- Update the object tracker using the detected regions.
- Assign or maintain tracking information for moving objects.
- Draw bounding boxes around tracked objects.

5.11 **ELSE**

- Continue tracking existing objects if they are temporarily unavailable.

5.12 Update the background model to adapt to changes in the dynamic background.

5.13 Display the current frame with motion regions and tracked objects highlighted.

5.14 Check for the termination command.

5.15 **IF** termination command is received **THEN**

- Exit the loop.

6. **END WHILE**

7. Release the video capture.

8. Destroy all display windows.

END

Conclusion

The proposed pseudocode provides a structured approach for a real-time motion detection and tracking system. It performs frame differencing, filtering, motion-region identification, object tracking, and visualization while continuously processing the video stream. The use of background updating helps the system adapt to dynamic background changes, and efficient frame processing supports near-real-time operation.

2. Facial Expression Recognition System

A mobile-based system must:

- A. Capture images from a camera
- B. Detect faces
- C. Extract facial features
- D. Classify expressions (happy, sad, neutral)
- E. Display results

Constraints:

- Limited computational resources
- Real-time response required

Write pseudocode for the pipeline, ensuring efficient processing and modular design for detection, feature extraction, and classification.

Input: Live video stream from a mobile camera

Output: Detected facial expression displayed on the screen

Pseudocode

BEGIN

1. Initialize the mobile camera.
2. Load a lightweight face detection model.
3. Load the facial feature extraction and expression classification model.
4. Define the expression classes:
 - Happy
 - Sad
 - Neutral

5. **WHILE** the camera is active **DO**

5.1 Capture the current image frame.

5.2 Resize the frame to a suitable size for efficient processing.

5.3 Convert the frame into the required format for face detection.

5.4 Detect all faces present in the frame.

5.5 **IF** a face is detected **THEN**

- Extract the Region of Interest (ROI) corresponding to each face.
- Normalize the extracted face region.

- Extract relevant facial features.
- Pass the features to the expression classifier.
- Predict the expression as **Happy, Sad, or Neutral**.
- Display the predicted expression on the corresponding face.

5.6 ELSE

- Display "No face detected".

5.7 Display the processed frame on the mobile screen.

5.8 Check whether the camera should be stopped.

5.9 IF stop command is received THEN

- Exit the loop.

6. **END WHILE**

7. Release the camera resources.

8. Close the display.

END

Conclusion

The proposed pseudocode provides a modular and efficient pipeline for real-time facial expression recognition on a mobile device. It performs image capture, face detection, facial feature extraction, expression classification, and result visualization in a continuous loop. The use of resized frames and a lightweight processing approach helps reduce computational requirements while maintaining the required real-time response.

3. Face Detection with ROI Extraction

A system must:

- A. Detect faces in an image
- B. Extract face regions
- C. Save cropped faces

Constraints:

- Handle multiple faces
- Avoid redundant processing

Write pseudocode including detection, ROI extraction, and saving logic.

Input: Image containing one or more faces

Output: Cropped face images saved separately

Pseudocode

BEGIN

1. Load the input image.
2. Initialize the face detection model.
3. Convert the input image into the required format for detection.
4. Detect all faces present in the image.
5. Initialize a counter for saving cropped faces.
6. **IF** no faces are detected **THEN**
 - Display "No faces detected".
 - Stop the process.
7. **ELSE**

FOR EACH detected face:

7.1 Obtain the bounding coordinates of the face:

- X-coordinate
- Y-coordinate
- Width
- Height

7.2 Check whether the detected face has already been processed.

7.3 **IF** the face has already been processed **THEN**

- Skip the face to avoid redundant processing.

7.4 **ELSE**

- Extract the face Region of Interest (ROI) using the bounding coordinates.
- Validate that the ROI is within the image boundaries.
- Crop the face region from the original image.
- Increment the face counter.
- Save the cropped face using a unique filename.
- Mark the detected face as processed.

8. **END FOR**

9. Display the number of faces detected and saved.

10. Release the image-processing resources.

END

Conclusion

The proposed pseudocode provides a structured method for detecting and extracting multiple faces from an image. It identifies each face using bounding coordinates, extracts the corresponding Region of Interest, checks for redundant processing, and saves each cropped face separately. This approach ensures that multiple faces are handled systematically while avoiding unnecessary repeated processing.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient